

深圳大学 计算机与软件学院

College of Computer Science and Software Engineering of Shenzhen University



系统编程

基于TaiShan服务器/openEuler OS 的实践

第一讲：课程介绍 & 进程控制（上）

为什么要学习系统编程？

1. 产业需求

【迅雷网络内推-社招】一大波高薪岗位来袭

岗位要求：

1. 本科及以上学历；
2. 3年以上linux后台开发相关工作经验；
3. 熟悉linux内核及调试方法，对linux内核各基础模块有比较深入理解；
4. 精通linux内核TCP/IP协议栈，对操作系统和体系结构有深刻的理解；
5. 熟悉linux内核常用工具链，对传输领域的新动向有敏锐的嗅觉。

存储工程师

北京研发类全职

工作职责：

1. 负责分布式存储后台开发，前端sdk开发。

岗位要求：

1. 本科及以上学历，计算机相关专业学历；
2. 精通C/C++，PHP和GO任意一种语言，熟悉Linux/Unix平台开发，至少熟悉一种脚本语言；
3. 精通TCP/IP与HTTP等网络协议相关知识，精通网络编程和多线程/多进程开发；
4. 熟悉ipfs、Sia优先；
5. 熟悉mysql、tair、leveldb、redis中一种或多种；
6. 熟悉Nginx、squid、vanish等相关技术，阅读过相关的内核代码实现；
7. 有Nginx模块开发或者服务端Lua开发经验；
8. 工作认真踏实、乐于且易于沟通，较强的学习能力、接受新事物的能力，以及对技术的热爱。

北京腾讯研究院招聘Linux后台开发工程师

TOP 6

工作类型：实习

工作地点：北京

企业性质：私营·股份制企业

所属行业：互联网·电子商务行业

薪资待遇：---

所属职类：软件开发工程师

岗位职责：

Linux后台开发工程师 - 实习生

职位描述：

从事无线业务后台开发工作

应聘条件：

- 全日制大学计算机及软件相关专业硕士或博士
- 能熟练使用Linux开发工具
- 熟悉C/C++语言基础，了解基本SHELL编程
- 熟悉TCP/IP协议族，了解SOCKET编程接口
- 熟悉多进程、多线程编程，了解其内部原理
- 具备良好的数据结构功底
- 充满激情，责任心强，良好的对外沟通和团队协作能力
- 能保证6个月以上的实习期

【社招】百度智能云计算部各种岗位内推，长期有效

Linux内核研发工程师

工作职责

- 针对服务的需求定制Linux内核，结合业务需求开发内核新功能
- 结合服务特点对Linux内核进行性能调优
- 内核新技术的研究和应用

任职要求

- 精通Linux内核，至少阅读过内核中的主要模块(文件系统，TCP/IP，I/O，内存管理)之一的源代码
- 熟悉Linux平台上的C语言编程，熟悉多进程多线程编程，熟悉socket编程
- 优秀的分析问题和解决问题的能力，对解决具有挑战性问题充满激情
- 了解主流内核&虚拟化技术(Xen、KVM、QEMU、libvirt等)的实现，阅读过相关源代码
- 善于学习新的知识，动手能力强，有进取心
- 良好的团队合作精神，较强的沟通能力
- 优秀的分析问题和解决问题的能力，对解决具有挑战性问题充满激情
- 有一定硬件架构设计经验者优先(资深研发工程师)

为什么要学习系统编程？

1. 产业需求

2. 计算机系统能力人才的需求

■ 快速实现原型系统的能力

- 需求分析、设计、实现、部署、运行和测试
- 程序性开发能力
- “系统性设计能力”

■ 设计高效程序的能力

- 多样化复杂应用
- 异构系统平台
- 辅助技术和工具

DeepSeek-R1本地版

temperature(对话的多样性)

0.60

0.01

1.00

top_p(采样率)

0.90

0.01

1.00

开始新对话



请问大模型时代，学习基于操作系统多进程编程、进程间通信、多线程编程等能提升就业竞争力吗？



思考内容

嗯，用户问的是在大模型时代，学习操作系统相关的多进程编程、进程间通信、多线程编程这些技能是否有助于提升就业竞争力。首先，我需要理解大模型时代的特点。大模型通常指像GPT-4这样的AI模型，它们需要大量的计算资源和高效的并行处理能力。这种情况下，底层系统优化变得尤为重要。

接下来，用户提到的多进程、进程间通信、多线程都属于系统级编程，这些技术在处理高并发、资源管理方面非常关键。大模型的训练和推理可能涉及分布式计算，这时候进程间通信和线程管理就显得很重要了。比如，在分布式训练中，不同节点之间的数据交换需要高效的IPC机制，而多线程可以充分利用多核CPU资源，提高处理速度。

然后，要考虑就业市场。虽然现在很多应用开发可能使用高级框架，但底层系统开发仍然需要这些技能。特别是在云计算、大数据、AI基础设施等领域，掌握这些知识的人才需求较大。例如，优化AI模型的推理速度可能需要深入操作系统层面的调优，这时候相关经验就很有竞争力。

DeepSeek-R1本地版

temperature(对话的多样性)

0.60

0.01

1.00

top_p(采样率)

0.90

0.01

1.00

开始新对话

另外，用户可能担心这些传统技术是否会被更高级的工具取代。但实际上，高级工具往往建立在底层技术之上。了解底层机制可以帮助开发者更好地解决问题，进行性能优化，这在复杂系统中是不可替代的。

还要考虑用户可能的背景。如果用户是学生或刚入行的开发者，学习这些基础技能可以打下坚实的基础，拓宽职业选择范围。对于有经验的开发者，深化系统级知识可能有助于转向更高阶的职位，如系统架构师或性能工程师。

最后，总结这些点，虽然大模型时代强调AI和数据处理，但底层系统编程仍然是支撑这些技术的关键。掌握这些技能不仅能提升就业竞争力，还能增强解决复杂问题的能力，适应更多技术挑战。

DeepSeek-R1本地版

temperature(对话的多样性)

0.60

0.01

1.00

top_p(采样率)

0.90

0.01

1.00

开始新对话

在大模型时代，学习基于操作系统的多进程编程、进程间通信（IPC）、多线程编程等底层技术仍然能显著提升就业竞争力。以下是具体原因：

1. 大模型与底层系统的高效协同

- 大模型（如GPT、LLM）的训练和推理依赖分布式计算、并行处理和高性能计算（HPC），需要深入理解多进程/线程管理以优化资源分配。
- 例如，分布式训练中跨节点通信（如GPU间数据传输）依赖高效的IPC机制（如共享内存、消息队列）。

2. 云计算与分布式系统的核心需求

- 云计算平台（如AWS、Kubernetes）和分布式系统（如Spark、Flink）的核心功能依赖于多进程/线程技术。
- 掌握这些技能可胜任云原生开发、分布式系统优化等岗位，尤其在大模型部署和推理加速领域需求旺盛。

DeepSeek-R1本地版

temperature(对话的多样性)

0.60

0.01

1.00

top_p(采样率)

0.90

0.01

1.00

开始新对话

3. 性能优化的不可替代性

- 大模型推理的实时性要求高，需通过**多线程并发**减少延迟，或通过**多进程隔离**提升稳定性。
- 例如，AI推理框架（如TensorRT、ONNX Runtime）的底层实现依赖线程池和进程管理。

4. 跨领域技术栈的通用性

- 多进程/线程技术是操作系统、嵌入式系统、游戏开发、实时系统等领域的通用技能，就业选择更广。
- 即使转向AI应用开发，理解底层机制有助于调试性能瓶颈（如GIL锁、死锁问题）。

5. 企业招聘的隐性门槛

- 大厂对候选人的**系统编程能力**要求严格，尤其是核心部门（如数据库、存储、AI基础设施）。
- 面试中常考察多线程同步（锁、信号量）、进程调度算法等题目，相关经验能通过简历筛选和技术面。

总结

尽管大模型时代强调AI算法和框架，但**系统级编程能力**是支撑上层技术的基石。学习多进程/线程编程不仅能提升竞争力，还能增强对复杂系统的掌控力，适应从AI基础设施到边缘计算等多样化场景的需求。

DeepSeek-R1本地版

temperature(对话的多样性)

0.60

0.01

1.00

top_p(采样率)

0.90

0.01

1.00

开始新对话



请问大模型时代，学习基于操作系统多进程编程、进程间通信、多线程编程等能提升科研创新能力吗？哪些科研、应用创新需要这些技术呢？



思考内容

在大模型时代，学习**多进程编程**、**进程间通信（IPC）**和**多线程编程**等操作系统底层技术，不仅能提升科研创新能力，还为许多前沿领域的突破提供了技术支撑。以下是具体分析：

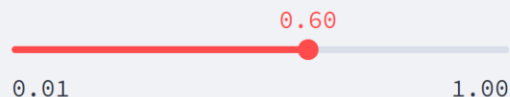
一、为何这些技术能提升科研创新能力？

1. 深入理解计算资源的分配与调度

大模型训练依赖分布式计算框架（如PyTorch Distributed、Horovod），其底层实现涉及多进程协作（如数据并行）和高效通信（如NCCL库）。掌握这些技术可优化资源利用率，设计更高效的分布式训练策略。

DeepSeek-R1本地版

temperature(对话的多样性)



top_p(采样率)



开始新对话

2. 突破单机算力瓶颈

大模型的参数量和数据量常超出单机处理能力，需通过多进程/线程技术实现跨节点并行（如模型并行、流水线并行）。例如，Meta的LLAMA训练使用**流水线并行**技术，依赖进程间通信协调多GPU计算。

3. 提升系统级调试能力

科研中常遇到死锁、资源竞争、内存泄漏等问题，熟悉多线程同步（如锁、信号量）和进程隔离机制可快速定位问题，加速实验迭代。

二、哪些科研方向依赖这些技术？

1. 分布式训练框架优化

- 目标：减少通信开销、提升训练效率。
- 案例：微软DeepSpeed通过**零冗余优化器（ZeRO）**在多进程中高效分配显存，依赖精细的进程间通信设计。

DeepSeek-R1本地版

temperature(对话的多样性)

0.60

0.01

1.00

top_p(采样率)

0.90

0.01

1.00

开始新对话

2. 边缘计算与联邦学习

- 场景：在资源受限设备（如手机、IoT）上部署模型，需通过多进程隔离计算任务，并通过IPC实现跨设备协作。
- 应用：谷歌的联邦学习框架使用多进程管理本地训练，并通过加密通信聚合模型参数。

3. 实时推理与低延迟系统

- 需求：大模型推理需高并发处理请求（如ChatGPT接口），通过线程池、异步IO等技术降低延迟。
- 案例：NVIDIA Triton推理服务器利用多线程并行处理请求，支持毫秒级响应。

4. AI与系统交叉研究

- 方向：
 - 编译优化：TVM、MLIR等编译器通过多线程加速计算图优化。
 - 硬件协同设计：为AI芯片（如TPU、NPU）设计定制化的进程调度策略，最大化硬件利用率。

DeepSeek-R1本地版

temperature(对话的多样性)

0.60

0.01

1.00

top_p(采样率)

0.90

0.01

1.00

开始新对话

三、应用创新场景示例

1. 多模态大模型实时交互

在机器人控制中，视觉、语音、运动规划模块可能以独立进程运行，通过共享内存或消息队列实现低延迟数据交换。

2. 科学计算与AI结合

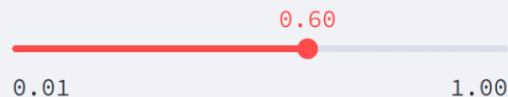
气候模拟或分子动力学中，将传统HPC（多进程MPI）与AI模型（如扩散模型）结合，加速仿真过程。

3. 隐私保护推理

使用安全多方计算（MPC）时，数据分片由多个进程处理，依赖加密通信协议确保隐私。

DeepSeek-R1本地版

temperature(对话的多样性)



top_p(采样率)



开始新对话

四、如何有效学习这些技术？

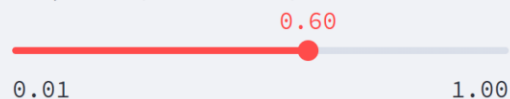
- **实践驱动**：从实现一个简单的多进程训练框架（如PyTorch多节点训练）入手，理解通信原语（AllReduce、Broadcast）。
- **开源项目参与**：贡献到分布式框架（如Ray、Dask）或推理引擎（如ONNX Runtime），接触真实场景的并发问题。
- **论文复现**：阅读系统顶会（OSDI、SOSP）中AI与系统结合的论文（如PipeDream的流水线并行），复现其多进程设计。

总结

操作系统底层技术是**连接算法创新与硬件效能**的桥梁。在大模型科研中，它们不仅是实现创新的工具，更是突破性能边界的关键。无论是优化训练效率、设计新型推理系统，还是探索AI与HPC的融合，这些技能都将成为科研竞争力的核心要素。

DeepSeek-R1本地版

temperature(对话的多样性)



top_p(采样率)



开始新对话

在大模型时代，学习**多进程编程**、**进程间通信**和**多线程编程**等操作系统技术对提升科研创新能力至关重要。以下是关键点总结：

1. 科研价值

- 优化分布式训练框架（如减少通信开销）。
- 实现边缘计算和联邦学习的跨设备协作。
- 设计低延迟推理系统（如线程池、异步IO）。

2. 创新应用场景

- **多模态实时交互**：机器人多进程协同。
- **AI+HPC融合**：气候模拟与模型联合加速。
- **隐私计算**：安全多方计算中的进程隔离。

3. 学习建议

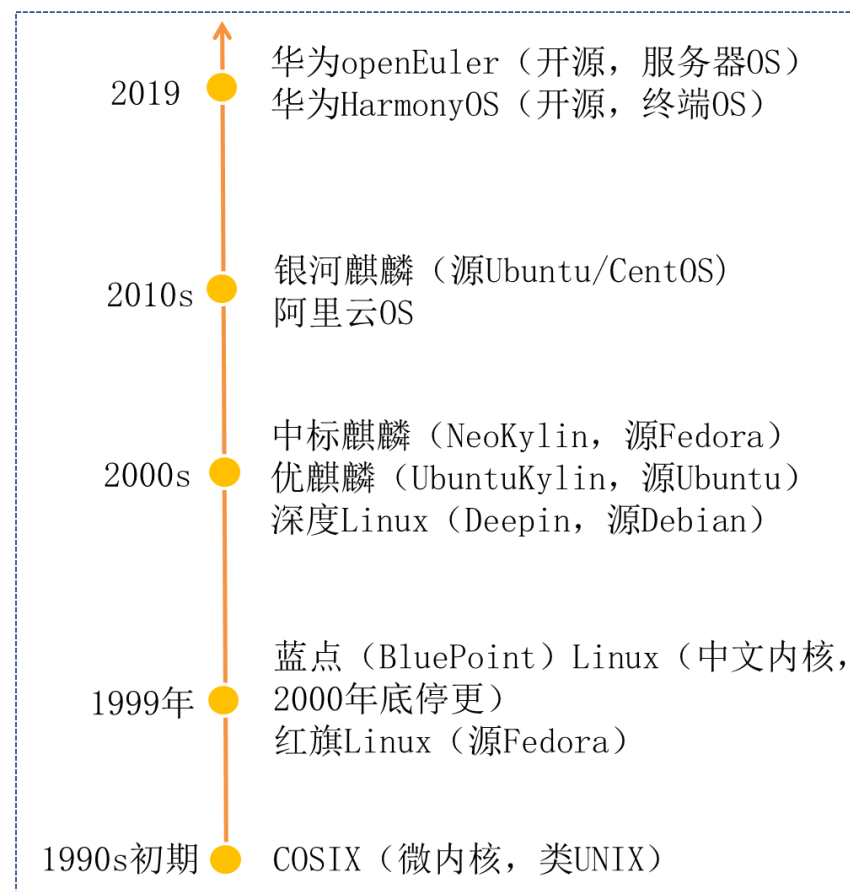
- 通过开源项目（Ray、ONNX Runtime）实践多进程/线程技术。
- 复现顶会论文中的系统设计（如PipeDream流水线并行）。

掌握这些技术不仅能解决大模型训练中的性能瓶颈，还为跨领域（如科学计算、隐私计算）的创新提供了底层支撑，是科研突破的核心能力之一。

为什么Taishan服务器 + openEuler操作系统?



(a) 操作系统发展进程



(a) 国产操作系统发展进程

华为: 服务器处理器 + 终端设备 + OS,
硬件产品的更新 - > OS的升级

闭环生态

<https://cloud.tencent.com/developer/news/530356>

课程内容

本课程围绕构建一个基于主机系统的**多客户即时通讯/聊天室**项目展开

- 理论知识

- ◆ 进程控制： 僵尸进程/孤儿进程、进程控制、守护进程。。。
- ◆ 进程间通信： 管道、命名管道、信号。。。
- ◆ 多线程编程： 锁、信号量、线程安全
- ◆ 文件操作： 系统结构、常用I/O系统调用、打开文件的内核表达…

- 4个基本实验

- ◆ 进程控制
- ◆ 进程间通信
- ◆ 多线程编程
- ◆ 文件操作

- 期末大作业 （任选其一完成）

- 多客户即时通讯软件： 课堂所学的OS系统调用+自学系统调用
- OS 开源社区贡献： Linux kernel/openEuler/Tencent OS……
- 前沿论文复现： 阅读系统顶会（OSDI、SOSP等）中AI与系统结合的论文（如PipeDream的流水线并行），复现其多进程、进程间通信、多线程编程设计

详细授课内容安排

操作系统考研知识点

第1-4周	课程介绍+进程控制
第5周	守护进程创建
第6-9周	进程间通信
第10-13周	线程控制
第14-16周	文件控制

一、操作系统概述 (一)操作系统的基本概念 (二)操作系统的发展 (三)程序运行环境

1. CPU 运行模式: 内核模式, 用户模式。 2. 中断和异常的处理

3. 系统调用

4. 程序的链接与装入

5. 程序运行时内存映像与地址空间 (四)操作系统结构

分层, 模块化, 宏内核, 微内核, 外核。 (五)操作系统引导

(六)虚拟机 二、进程管理

(一)进程与线程 1. 进程概念

2. 进程的状态与转换

3. 线程的实现: 内核支持的线程, 线程库支持的线程。 4. 进程与线程的组织与控制

5. 进程间通信: 共享内存, 消息传递, 管道。

(二)CPU 调度与上下文切换 1. 调度的基本概念

2. 调度的目标

3. 调度的实现: 调度器/调度程序(scheduler), 调度的时机与调度方式(抢占式/非抢占式), 就绪进程, 内核级线程与用户级线程调度。

4. 典型调度算法

先来先服务调度算法; 短作业(短进程、短线程)优先调度算法; 时间片轮转调度算法; 优先级调度算法;

高响应比优先调度算法; 多级反馈队列调度算法。

5. 上下文及其切换机制 (三)同步与互斥

1. 进程同步的基本概念

2. 实现临界区互斥的基本方法: 软件实现方法; 硬件实现方法。 3. 锁

4. 信号量

5. 条件变量

6. 经典同步问题

生产者-消费者问题; 读者-写者问题; 哲学家进餐问题。

(四)死锁

1. 死锁的概念

2. 死锁预防

3. 死锁避免

4. 死锁检测和解除

三、内存管理 (一)内存管理基础

1. 内存管理的基本概念: 程序装入与链接; 逻辑地址与物理地址空间; 内存保护。内存分配与回收

2. 连续分配管理方式

3. 分页管理方式

4. 分段管理方式

5. 分页式管理方式。 (二)虚拟内存管理

1. 虚拟内存基本概念

2. 请求分页管理方式

3. 页框分配

4. 页面置换算法: 最佳置换算法(OPT); 先进先出置换算法(FIFO); 最近最少使用置换算法(LRU); 时钟置换算法(CLOCK)。 5. 内存映射文件(Memory-Mapped Files)

6. 虚拟存储器性能的影响因素及改进方法 四、文件管理

(一)文件系统基础 1. 文件概念

2. 文件元数据和索引节点

3. 文件的操作: 建立, 删除, 打开, 关闭, 读, 写。 4. 文件共享

5. 文件的逻辑结构

6. 文件的物理结构 (二)目录

1. 目录的基本概念

2. 树形目录

3. 目录的操作

4. 硬链接与软链接 (三)文件系统

1. 文件系统的全局结构(layout) 文件系统在外存中的结构, 文件系统在内存中的结构。 2. 外存空闲空间管理方法

3. 虚拟文件系统 4. 文件系统挂载(mounting)

五、输入输出(I/O)管理 (一)I/O 管理基础

1. 设备

设备的基本概念, 设备的分类, I/O 接口, I/O 端口。 2. I/O 控制方式

轮询方式, 中断方式, DMA 方式。 3. I/O 软件层次结构

中断处理程序, 驱动程序, 设备独立软件, 用户层 I/O 软件。 4. 输入输出应用程序接口 字符设备接口

块设备接口, 网络设备接口, 阻塞/非阻塞 I/O。

(二)设备独立软件

1. 缓冲区管理

2. 设备分配与回收

3. 假脱机技术(SPOOLing)

4. 设备驱动程序接口 (三)外存管理

1. 磁盘: 磁盘结构, 格式化, 分区, 磁盘调度方法。 2. 固态硬盘

读写性能特性, 磨损均衡。

成绩说明



平时成绩

实验: 4次

闭卷考试: 1次

课堂表现: 加分, 代码分享, 问题探讨

总成绩

平时成绩 - 40%

大作业 - 60%

课上一分钟, 课后几天功

- ✓ 相关概念及例子讲解
- ✓ 课堂练习、程序调试与典型错误分析
- ✓ 随堂实验答疑: 知无不言, 言无不尽
- ✓ 作业讲评: 巩固理解, 加深印象
- ✓ 闭卷考试: 国内外考试题目, 原理为主

参考资料

■ 教材

- 建设中

■ 参考教材

- W. Richard Stevens, Stephen A. Rago, UNIX环境高级编程（第3版），尤晋元 张亚英 戚正伟 译，人民邮电出版社
- Mark L. Mitchell, Alex Samuel, Jeffrey Oldham, Advanced Linux Programming , New Riders
- Robert Love, Linux System Programming, O'Reilly Taiwan公司译, 东南大学出版社出版
- C语言编程实战

■ 参考课件

- 由尚德 - Linux 系统编程
- 宋健健 - Linux 程序设计

■ openEuler + glibc 相关源代码

关于4次作业的展示

量大

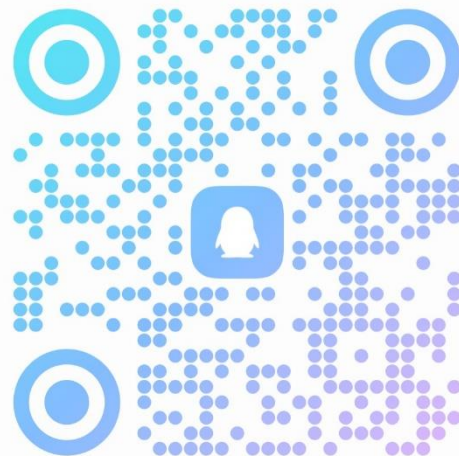
题难

非喜勿进



2025系统编程

群号: 1018049749



扫一扫二维码，加入群聊



本节课内容

区分OS代码和用户代码

程序编译与调试

进程的基本概念

进程控制相关的系统调用

`fork()`,

`getpid()`, `getppid()`

`exit()`

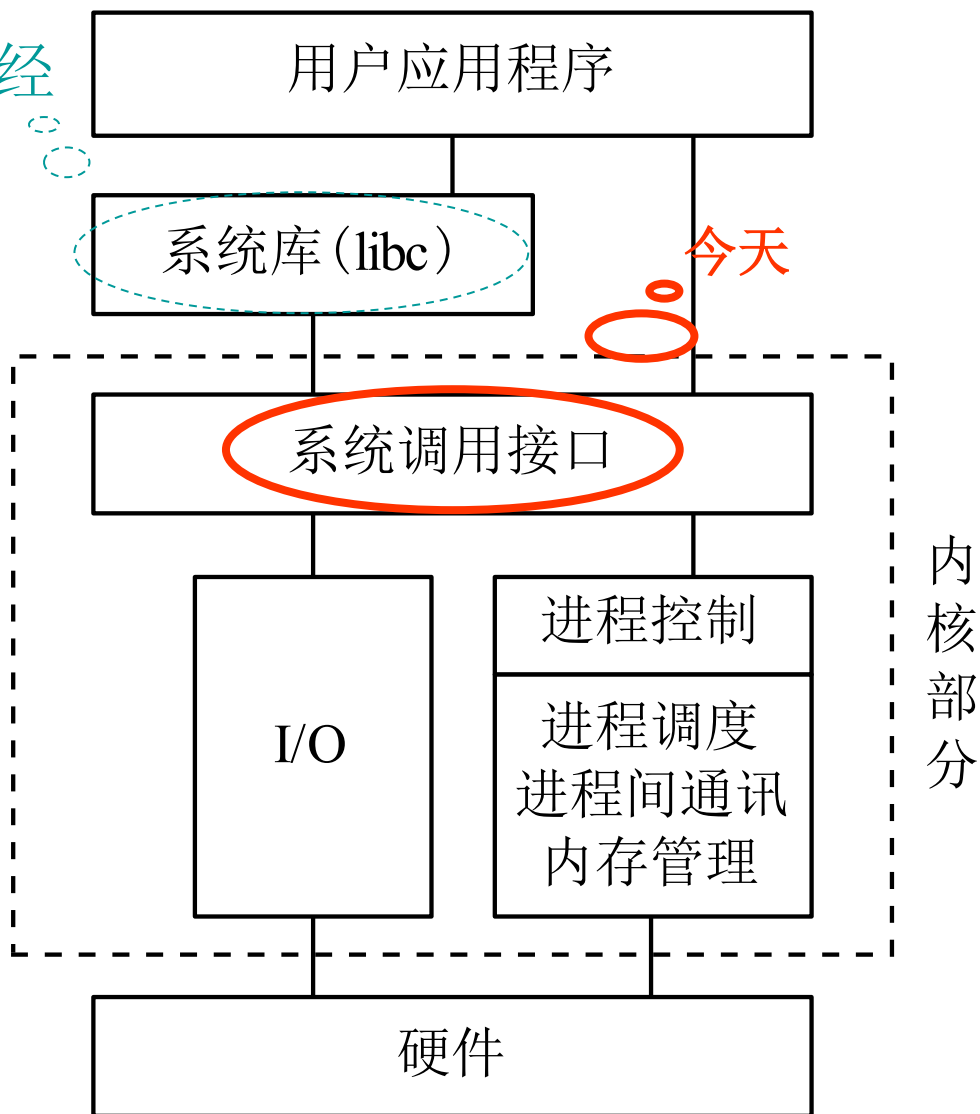
`wait()`, `waitpid()`

库函数和OS核心代码的关系

■ 库函数调用OS代码来完成他们的工作

- `printf(3)`
 - 解释输入
 - 格式化输入
 - 调用 `write(2)`

曾经



应用软件 vs. 系统软件

应用软件	系统软件
为客户提供服务，如QQ音乐播放器	为应用提供服务，如资源组织、管理、调度和使用
处理人们关心的问题，如医疗保健、游戏、金融等	控制和管理计算机系统资源
关注功能、用户体验等	关注效率，软硬件协同优化
独立于设备或平台，使用通用算法	具体情况具体分析，可依赖于特定设备驱动、操作系统等

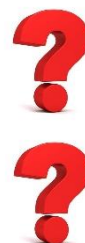
Topics in Systems Programming

This list isn't intended to be complete, just informative

- The nature of data and computation
- What a computer can do — and can do *efficiently*
- The organization and components of a computer system
- How data is encoded and decoded
- Digital Logic
- The C Programming Language (and others like it)
- Assembly and Machine Languages
- Basic operating system functionality (file, process, and memory management)
- Interfacing with the O.S.
- Compiling, assembling, linking, loading
- Exploiting systems
- Defending systems from attack
- File formats
- Advanced operating system functionality (virtual memory, interrupts, bootstrapping)
- Device Drivers
- Concurrency basics: threads and events
- Networking basics

系统调用和库函数之间的区别

系统调用	库函数
操作系统提供	LibC标准函数库、Java库函数、C++标准模板库
联机帮助第2节	联机帮助第3节
依赖于系统	依赖于编程语言
高效	较高效
运行在内核空间	运行在用户空间



课程结束后可再回来自己总结下

程序编译与调试 - 代码

```
#include <errno.h>
#include <sys/stat.h>
#include <sys/fcntl.h>
int main()
{
    int fd;
    char *myc = "This is my 1st program!!\n";
    char *myf = "myfile";
    if ((fd = open(myf,O_RDWR|O_CREAT|O_APPEND,0644)) == -1) {
        perror(myf);
        exit(errno);
    }
    if (write(fd,myc,strlen(myc)) != strlen(myc)){
        perror(myc);
        close(fd);
        exit(errno);
    }
    close(fd);
}
```

```
[szu@taishan02-vm-10 compileddebug]$ gcc -o useSystem useSystem.c
useSystem.c: In function 'main':
useSystem.c:10:4: warning: implicit declaration of function 'perror'; did you mean 'errno'? [-Wimplicit-function-declaration]
    perror(myf);
    ^~~~~~
    errno
useSystem.c:11:4: warning: implicit declaration of function 'exit' [-Wimplicit-function-declaration]
    exit(errno);
    ^~~~
useSystem.c:11:4: warning: incompatible implicit declaration of built-in function 'exit'
useSystem.c:11:4: note: include '<stdlib.h>' or provide a declaration of 'exit'
useSystem.c:13:6: warning: implicit declaration of function 'write' [-Wimplicit-function-declaration]
    if (write(fd,myc,strlen(myc)) != strlen(myc)){
        ^~~~~~
useSystem.c:13:19: warning: implicit declaration of function 'strlen' [-Wimplicit-function-declaration]
    if (write(fd,myc,strlen(myc)) != strlen(myc)){
                    ^~~~~~
useSystem.c:13:19: warning: incompatible implicit declaration of built-in function 'strlen'
useSystem.c:13:19: note: include '<string.h>' or provide a declaration of 'strlen'
useSystem.c:15:4: warning: implicit declaration of function 'close' [-Wimplicit-function-declaration]
    close(fd);
    ^~~~~~
useSystem.c:16:4: warning: incompatible implicit declaration of built-in function 'exit'
    exit(errno);
    ^~~~
useSystem.c:16:4: note: include '<stdlib.h>' or provide a declaration of 'exit'
[szu@taishan02-vm-10 compileddebug]$
```

- 重视编译警告信息
- 最好修改代码致0警告
- \$man exit
- \$man strlen

```
#include <errno.h>
#include <sys/stat.h>
#include <sys/fcntl.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <stdio.h>
int main()
{
    int fd;
    char *myc = "This is my 1st program!!\n";
    char *myf = "myfile";
    if ((fd = open(myf,O_RDWR|O_CREAT|O_APPEND,0644)) == -1) {
        perror(myf);
        exit(errno);
    }
    if (write(fd,myc,strlen(myc)) != strlen(myc)){
        perror(myc);
        close(fd);
        exit(errno);
    }
    close(fd);
}
```

```
[szu@taishan02-vm-10 compileddebug]$ gcc -o useSystem useSystem.c
[szu@taishan02-vm-10 compileddebug]$ ls
useSystem  useSystem.c
[szu@taishan02-vm-10 compileddebug]$ ./useSystem
[szu@taishan02-vm-10 compileddebug]$ ls
myfile  useSystem  useSystem.c
[szu@taishan02-vm-10 compileddebug]$ cat myfile
This is my 1st program!!
[szu@taishan02-vm-10 compileddebug]$
```

```

#include <errno.h>
#include <sys/stat.h>
#include <sys/fcntl.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <stdio.h>
int main()
{
    int fd;
    char *myc = "This is my 1st program!!\n";
    char *myf = "myfile";
    if ((fd = open(myf,O_RDWR|O_CREAT|O_APPEND,0644)) == -1) {
        perror(myf);
        exit(errno);
    }
    if (write(fd,myc,strlen(myc)) != strlen(myc)){
        perror(myc);
        close(fd);
        exit(errno);
    } else {
        system(strcat("cat ",myf));
    }
    close(fd);
}

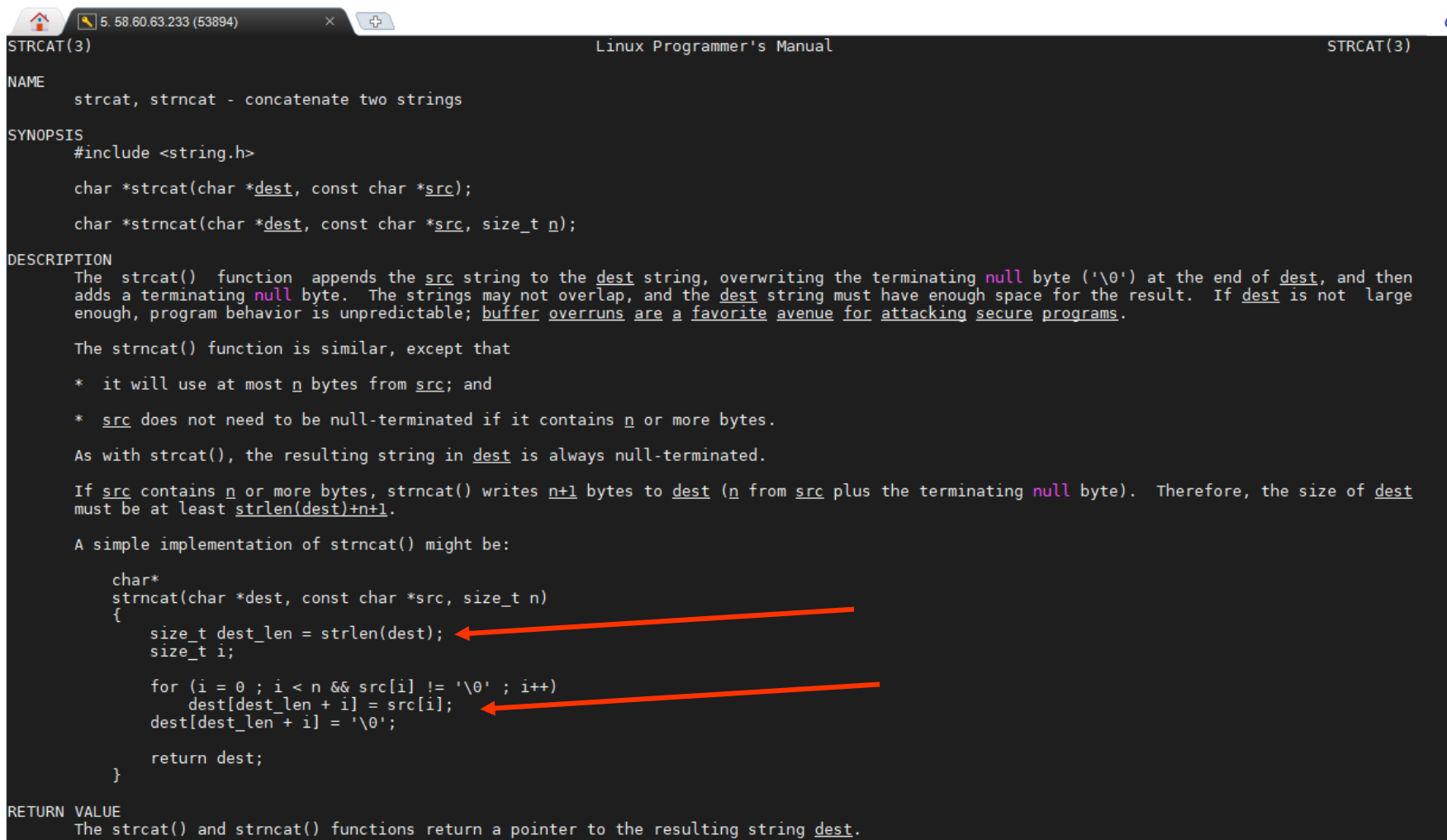
```

```

[szu@taishan02-vm-10 compileddebug]$ cp useSystem.c useSystem_1.c
[szu@taishan02-vm-10 compileddebug]$ vi useSystem_1.c
[szu@taishan02-vm-10 compileddebug]$ gcc -o useSystem_1 useSystem_1.c
[szu@taishan02-vm-10 compileddebug]$ ./useSystem_1
Segmentation fault (core dumped)
[szu@taishan02-vm-10 compileddebug]$ █

```

\$. /useSystem



```
STRCAT(3) Linux Programmer's Manual STRCAT(3)

NAME
    strcat, strncat - concatenate two strings

SYNOPSIS
    #include <string.h>

    char *strcat(char *dest, const char *src);

    char *strncat(char *dest, const char *src, size_t n);

DESCRIPTION
    The strcat() function appends the src string to the dest string, overwriting the terminating null byte ('\0') at the end of dest, and then adds a terminating null byte. The strings may not overlap, and the dest string must have enough space for the result. If dest is not large enough, program behavior is unpredictable; buffer overruns are a favorite avenue for attacking secure programs.

    The strncat() function is similar, except that
    * it will use at most n bytes from src; and
    * src does not need to be null-terminated if it contains n or more bytes.

    As with strcat(), the resulting string in dest is always null-terminated.

    If src contains n or more bytes, strncat() writes n+1 bytes to dest (n from src plus the terminating null byte). Therefore, the size of dest must be at least strlen(dest)+n+1.

    A simple implementation of strncat() might be:

    char*
    strncat(char *dest, const char *src, size_t n)
    {
        size_t dest_len = strlen(dest);
        size_t i;

        for (i = 0 ; i < n && src[i] != '\0' ; i++)
            dest[dest_len + i] = src[i];
        dest[dest_len + i] = '\0';

        return dest;
    }

RETURN VALUE
    The strcat() and strncat() functions return a pointer to the resulting string dest.
```

```

#include <errno.h>
#include <sys/stat.h>
#include <sys/fcntl.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <stdio.h>
int main()
{
    int fd;
    char *myc = "This is my 1st program!!\n";
    char *myf = "myfile";
    if ((fd = open(myf,O_RDWR|O_CREAT|O_APPEND,0644)) == -1) {
        perror(myf);
        exit(errno);
    }
    if (write(fd,myc,strlen(myc)) != strlen(myc)){
        perror(myc);
        close(fd);
        exit(errno);
    } else {
        char cmd[30] = "cat ";
        system(strcat(cmd,myf));
    }
    close(fd);
}

```

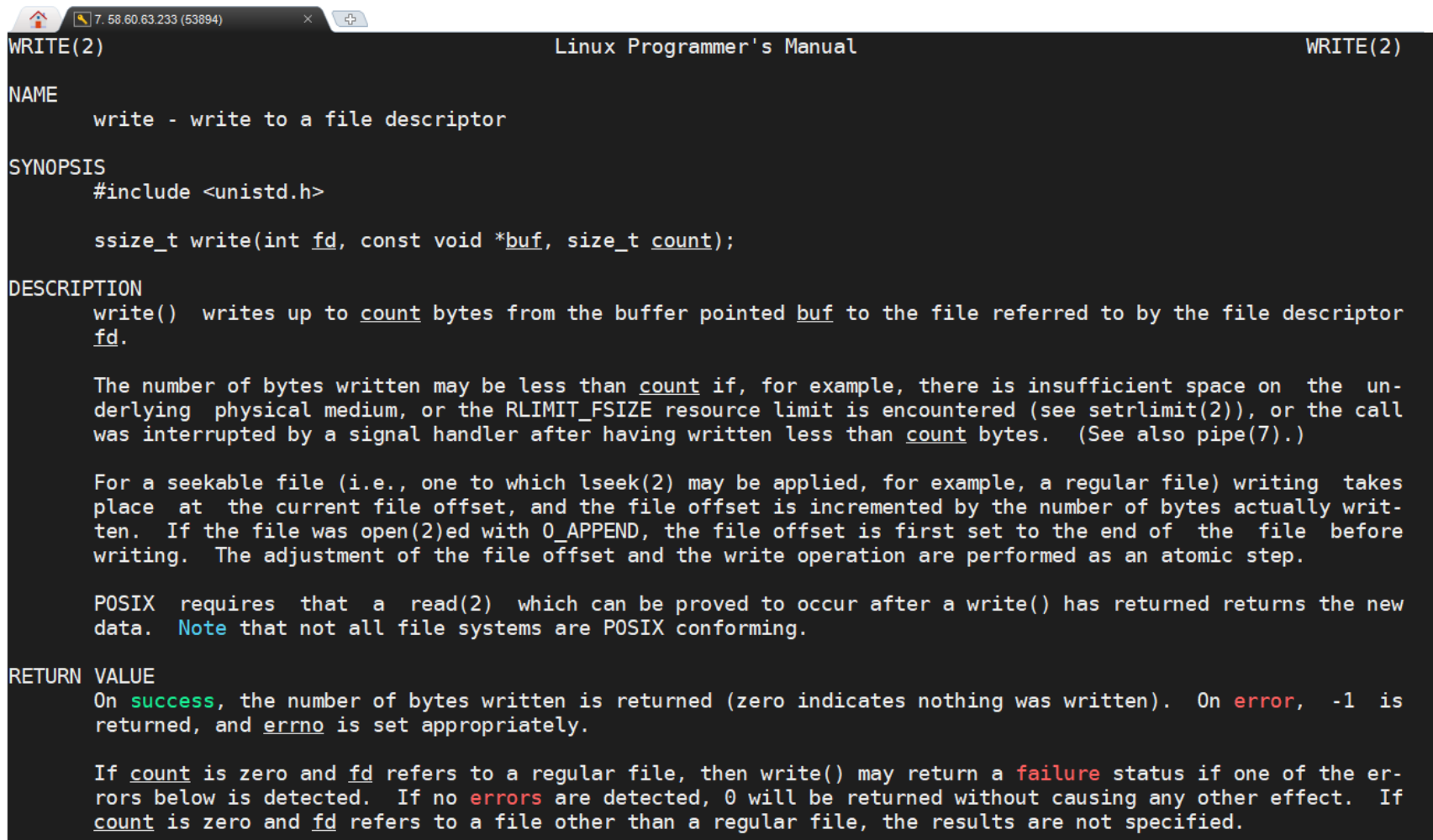
Perfect!

```

[szu@taishan02-vm-10 compileddebug]$ gcc -Wall -o useSystem_2 useSystem_2.c
[szu@taishan02-vm-10 compileddebug]$ ./useSystem_2
This is my 1st program!!
This is my 1st program!!
This is my 1st program!!
[szu@taishan02-vm-10 compileddebug]$ █

```

系统调用联机帮助使用例子 (1)



```
WRITE(2)                                     Linux Programmer's Manual                                     WRITE(2)

NAME
    write - write to a file descriptor

SYNOPSIS
    #include <unistd.h>

    ssize_t write(int fd, const void *buf, size_t count);

DESCRIPTION
    write() writes up to count bytes from the buffer pointed buf to the file referred to by the file descriptor fd.

    The number of bytes written may be less than count if, for example, there is insufficient space on the underlying physical medium, or the RLIMIT_FSIZE resource limit is encountered (see setrlimit(2)), or the call was interrupted by a signal handler after having written less than count bytes. (See also pipe(7).)

    For a seekable file (i.e., one to which lseek(2) may be applied, for example, a regular file) writing takes place at the current file offset, and the file offset is incremented by the number of bytes actually written. If the file was open(2)ed with O_APPEND, the file offset is first set to the end of the file before writing. The adjustment of the file offset and the write operation are performed as an atomic step.

    POSIX requires that a read(2) which can be proved to occur after a write() has returned returns the new data. Note that not all file systems are POSIX conforming.

RETURN VALUE
    On success, the number of bytes written is returned (zero indicates nothing was written). On error, -1 is returned, and errno is set appropriately.

    If count is zero and fd refers to a regular file, then write() may return a failure status if one of the errors below is detected. If no errors are detected, 0 will be returned without causing any other effect. If count is zero and fd refers to a file other than a regular file, the results are not specified.
```

ERRORS

EAGAIN The file descriptor `fd` refers to a file other than a socket and has been marked `nonblocking` (`O_NONBLOCK`), and the write would block.

EAGAIN or **EWouldBlock**

The file descriptor `fd` refers to a socket and has been marked `nonblocking` (`O_NONBLOCK`), and the write would block. POSIX.1-2001 allows either **error** to be returned for this case, and does not require these constants to have the same value, so a portable application should check for both possibilities.

EBADF `fd` is not a valid file descriptor or is not open for writing.

EDESTADDRREQ

`fd` refers to a datagram socket for which a peer address has not been set using `connect(2)`.

EDQUOT The user's quota of disk blocks on the file system containing the file referred to by `fd` has been exhausted.

EFAULT `buf` is outside your accessible address space.

EFBIG An attempt was made to write a file that exceeds the implementation-defined maximum file size or the process's file size limit, or to write at a position past the maximum **allowed** offset.

EINTR The call was interrupted by a signal before any data was written; see `signal(7)`.

EINVAL `fd` is attached to an object which is unsuitable for writing; or the file was opened with the `O_DIRECT` flag, and either the address specified in `buf`, the value specified in `count`, or the current file offset is not suitably aligned.

EIO A low-level I/O **error** occurred while modifying the inode.

ENOSPC The device containing the file referred to by `fd` has no room for the data.

EPIPE `fd` is **connected** to a pipe or socket whose reading end is **closed**. When this happens the writing process will also receive a `SIGPIPE` signal. (Thus, the write return value is seen only if the program catches, blocks or ignores this signal.)

Other **errors** may occur, depending on the object **connected** to `fd`.

CONFORMING TO

SVr4, 4.3BSD, POSIX.1-2001.

Under SVr4 a write may be interrupted and return `EINTR` at any point, not just before any data is written.

NOTES

A **successful** return from `write()` does not make any guarantee that data has been committed to disk. In fact, on some buggy implementations, it does not even guarantee that space has **successfully** been reserved for the data. The only way to be sure is to call `fsync(2)` after you are done writing all your data.

If a `write()` is interrupted by a signal handler before any bytes are written, then the call fails with the **error** `EINTR`; if it is interrupted after at least one byte has been written, the call succeeds, and returns the number of bytes written.

SEE ALSO

`close(2)`, `fcntl(2)`, `fsync(2)`, `ioctl(2)`, `lseek(2)`, `open(2)`, `pwrite(2)`, `read(2)`, `select(2)`, `writev(2)`, `fwrite(3)`

Manual page `write(2)` line 34 (press h for help or q to quit)

出错时 - 返回值

■ 成功执行

- 返回非负文件描述符
- 返回指向对象的指针
- ◦ ◦

■ 运行出错

- 返回-1
- 返回null指针
- ◦ ◦

■ 只知道成功与否，如果出错，不知道原因

```
if ((fd = open("/proc/self/pagemap", O_RDONLY)) == -1) {  
    printf("open /proc/self/pagemap error\n");  
    exit(-1);  
}
```

But, must think about applications.
Not always **appropriate** to **exit** when
something goes wrong.

-- 《computer systems a
programmer perspective》

出错时 – 系统全局整型变量errno

■ POSIX定义: `extern int errno`

- 定义在头文件`/usr/include/asm-generic/errno.h`中
- `errno`常数以E开头
- `errno`值为非0值
- 按系统调用分类, 指向特定错误的值
 - ◆ 与open出错相关的的`errno`大约有15种

■ 使用规则

- 如果调用没有出错, `errno`值为上一个出错的调用所设置
- 当且仅当函数返回值指明出错时, `errno`值才有意义

出错时 - errno - 相关系统调用

■ strerror ()

- 功能：以字符串方式打印错误信息

- 用法：

```
#include <string.h>
```

```
char *strerror(int errnum);
```

- 返回：指向消息字符串的指针

■ perror ()

- 功能：打印说明错误的参数字符串和标准字符串信息

- 用法：

```
#include <stdio.h>
```

```
void perror(const char * msg);
```

- 返回：void

处理错误的示例

```
#include <errno.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/fcntl.h>
#include <string.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    char *myf="myfile";
    int fd = open(myf,O_RDWR);

    if(fd == -1){
        fprintf(stderr,"Error Msg: %s\n",strerror(errno));
        perror(myf);
        exit(errno);
    }
    printf("Open a file\n");
    return 0;
}
```

系统调用open() 出错有25个原因:

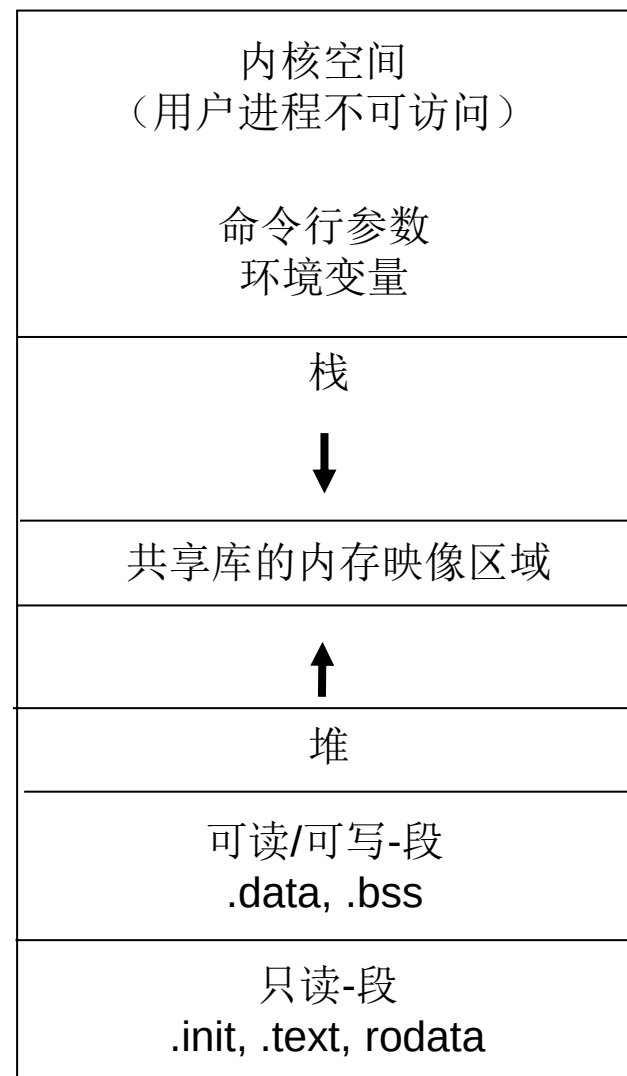
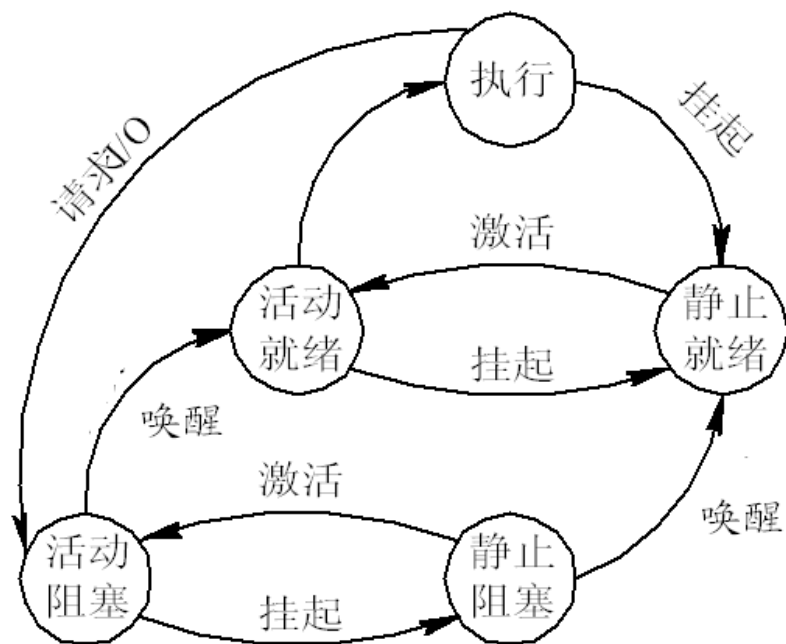
`$man 2 open | sed 's/[ ] [ ]*//g' | cut -d ' ' -f1 | grep ^E | wc -l`

进程

■ 进程是程序执行时的一个实例

- 内存映像
- 页表
- 物理内存
- PCB

■ 进程是系统资源分配和调度的实体



进程的状态

■ TASK_RUNNING (可执行状态)

- 正在执行
- 就绪

■ TASK_INTERRUPTIBLE (可中断, 睡眠/阻塞)

- 条件满足
 - 收到信号
- } 就绪

■ TASK_UNINTERRUPTIBLE (不可中断)

- 条件满足 -> 就绪

■ TASK_ZOMBIE (僵死)

- 该进程已结束
- 父进程尚未收集状态

■ TASK_STOPPED (停止)

- 收到SIGSTOP、SIGTSTP、SIGTTIN、SIGTTOU等信号
- 收到信号SIGCONT-> 就绪

```
/* Used in tsk->state: */
#define TASK_RUNNING          0x0000
#define TASK_INTERRUPTIBLE    0x0001
#define TASK_UNINTERRUPTIBLE  0x0002
#define __TASK_STOPPED        0x0004
#define __TASK_TRACED         0x0008
/* Used in tsk->exit_state: */
#define EXIT_DEAD              0x0010
#define EXIT_ZOMBIE           0x0020
#define EXIT_TRACE             (EXIT_ZOMBIE | EXIT_DEAD)
/* Used in tsk->state again: */
#define TASK_PARKED            0x0040
#define TASK_DEAD              0x0080
#define TASK_WAKEKILL          0x0100
#define TASK_WAKING            0x0200
#define TASK_NOLOAD            0x0400
#define TASK_NEW                0x0800
#define TASK_STATE_MAX        0x1000

/* Convenience macros for the sake of set_current_state: */
#define TASK_KILLABLE          (TASK_WAKEKILL | TASK_UNINTERRUPTIBLE)
#define TASK_STOPPED           (TASK_WAKEKILL | __TASK_STOPPED)
#define TASK_TRACED            (TASK_WAKEKILL | __TASK_TRACED)

#define TASK_IDLE               (TASK_UNINTERRUPTIBLE | TASK_NOLOAD)

/* Convenience macros for the sake of wake_up(): */
#define TASK_NORMAL             (TASK_INTERRUPTIBLE | TASK_UNINTERRUPTIBLE)

/* get_task_state(): */
#define TASK_REPORT             (TASK_RUNNING | TASK_INTERRUPTIBLE | \
    TASK_UNINTERRUPTIBLE | __TASK_STOPPED | \
    __TASK_TRACED | EXIT_DEAD | EXIT_ZOMBIE | \
    TASK_PARKED)
```

/usr/src/kernels/4.19.90-
2003.4.0.0036.oe1.aarch64/include/linux/sched.h

进程状态的查看

■ ps: 显示瞬间进程的状态

■ 常用参数:

l: 长格式输出

u: 按用户名和启动时间的顺序来显示进程

j: 用任务格式来显示进程

f: 用树形格式来显示进程

a: 显示所有用户的所有进程

x: 显示无控制终端的进程

r: 显示运行中的进程

ww: 避免详细参数被截断

进程状态的查看

■ ps使用范例

\$ps //列出当前shell里当前用户的进程

\$ps -u yuhong //列出用户yuhong运行的所有进程

\$ps -el //以详细列表方式显示运行的所有进程

\$ps aux //以详细的BSD风格显示运行的所有进程

```
[szu@taishan02-vm-10 ~]$ ps aux | more
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2 110336 19520 ?        Ss   5月17   1:22 /usr/lib/sys
temd/systemd --switched-root --system --deserialize 16
root           2  0.0  0.0      0     0 ?        S    5月17   0:01 [kthreadd]
root           3  0.0  0.0      0     0 ?        I<   5月17   0:00 [rcu_gp]
root           4  0.0  0.0      0     0 ?        I<   5月17   0:00 [rcu_par_gp]
root           6  0.0  0.0      0     0 ?        I<   5月17   0:00 [kworker/0:0
H-kblockd]
root           8  0.0  0.0      0     0 ?        I<   5月17   0:00 [mm_percpu_w
q]
root           9  0.0  0.0      0     0 ?        S    5月17   0:00 [ksoftirqd/0
]
root          10  0.0  0.0      0     0 ?        I    5月17   0:32 [rcu_sched]
```

%MEM: 物理内存占用率 VSZ : 虚拟内存大小

RSS: 实际分配给进程使用的物理内存

STAT: 进程当前状态

进程状态

```
[szu@taishan02-vm-10 ~]$ ps aux | more
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2 110336 19520 ?        Ss   5月17   1:22 /usr/lib/sys
temd/systemd --switched-root --system --deserialize 16
root           2  0.0  0.0      0     0 ?        S    5月17   0:01 [kthreadd]
root           3  0.0  0.0      0     0 ?        I<   5月17   0:00 [rcu_gp]
root           4  0.0  0.0      0     0 ?        I<   5月17   0:00 [rcu_par_gp]
root           6  0.0  0.0      0     0 ?        I<   5月17   0:00 [kworker/0:0
H-kblockd]
root           8  0.0  0.0      0     0 ?        I<   5月17   0:00 [mm_percpu_w
q]
root           9  0.0  0.0      0     0 ?        S    5月17   0:00 [ksoftirqd/0
]
root          10  0.0  0.0      0     0 ?        I    5月17   0:32 [rcu_sched]
```

■ TASK_RUNNING (运行) **R**

■ TASK_INTERRUPTIBLE (可中断) **S**

■ TASK_UNINTERRUPTIBLE (不可中断) **D**

■ TASK_ZOMBIE (僵死) **Z**

■ TASK_STOPPED (停止) **T**

■ 后缀

- < (高优先级进程)
- N (低优先级进程)
- L (内存锁页)
- s (会话首进程)
- + (前台进程)
- l (多线程进程)

进程状态为Ss1, 请描述进程状态

进程的id - getpid(), getppid()

■ 系统调用

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
pid_t getpid(void); //获取自己的pid
```

```
pid_t getppid(void); //获取父进程的pid
```

■ 并发进程最大数

- 默认设置：32768（short int的最大值）
- /proc/sys/kernel/pid_max : 可根据情况设置

■ 范例:

```
#include <stdio.h>
#include <unistd.h>

int main(){
    printf("Process has as ID the number: %ld \n", (long) getpid());
    printf("Parent of the Process has as ID: %ld \n", (long) getppid());
    return 0;
}
```

用户空间 vs. 内核空间

user space vs. kernel space

JULIA EVANS
@b0rk

drawings.jvns.ca

the Linux kernel has
millions of lines of code

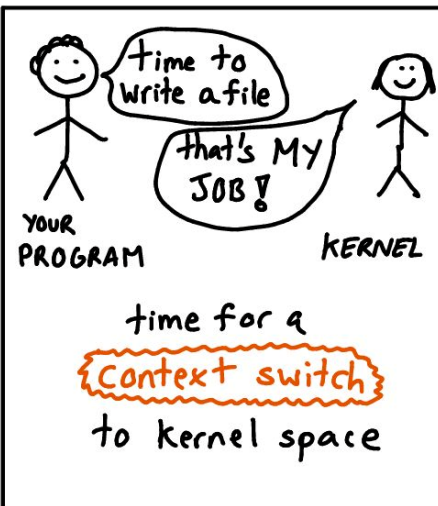
- ★ read+write files
- ★ decide which programs get to use the CPU
- ★ make the keyboard work

When Linux kernel code runs, that's called

kernel space

When your program runs, that's

user space



your program switches
back and forth

```
str = "my string"
```

```
x = x + 2
```

```
file.write(str) ← ★ switch to kernel space ★
```

```
y = x + 4
```

```
str = str * y ← ★ and we're back to user space! ★
```

timing your process

\$ time find /home

0.15 user

0.73 system

↑
time spent in
your process

↑
time spent by
the kernel doing
work for your
process

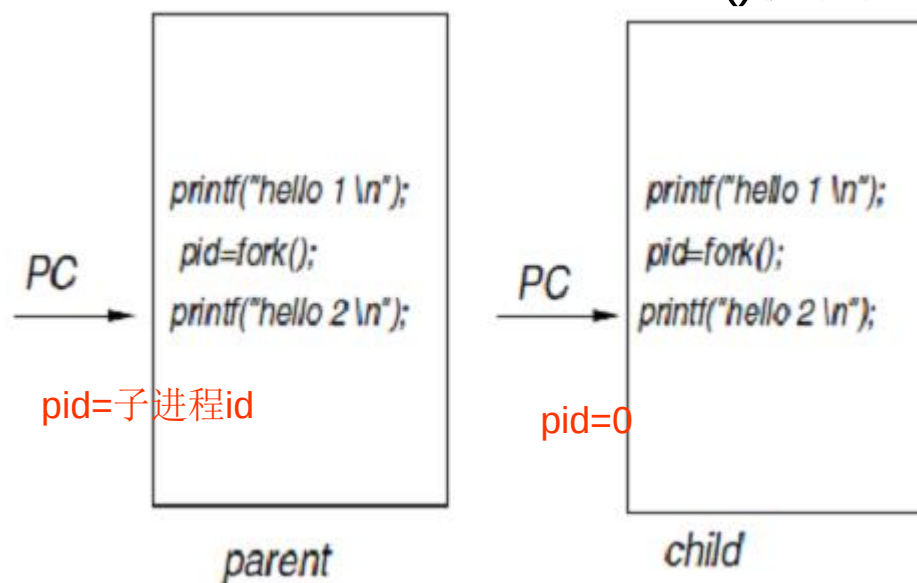
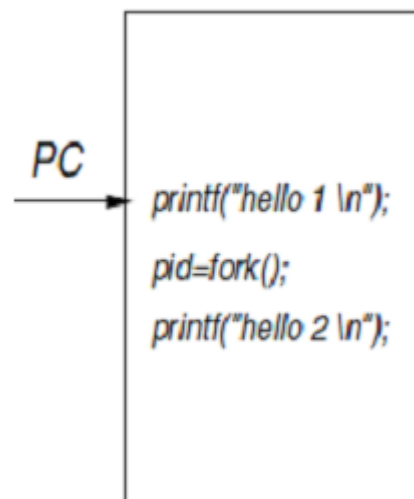
创建一个新的进程 - fork()

■ 语法

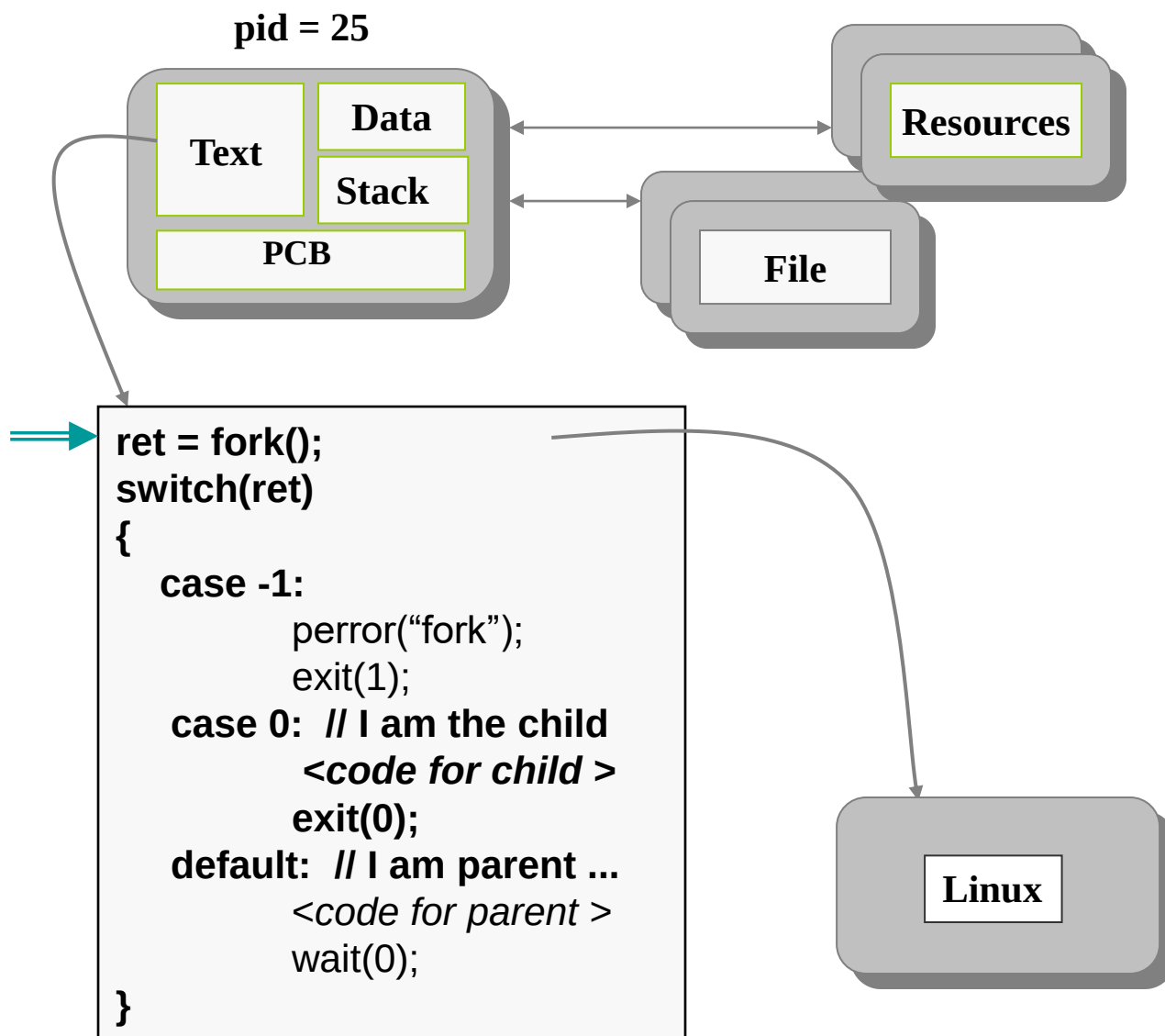
```
#include <unistd.h>

pid_t fork(void);
```

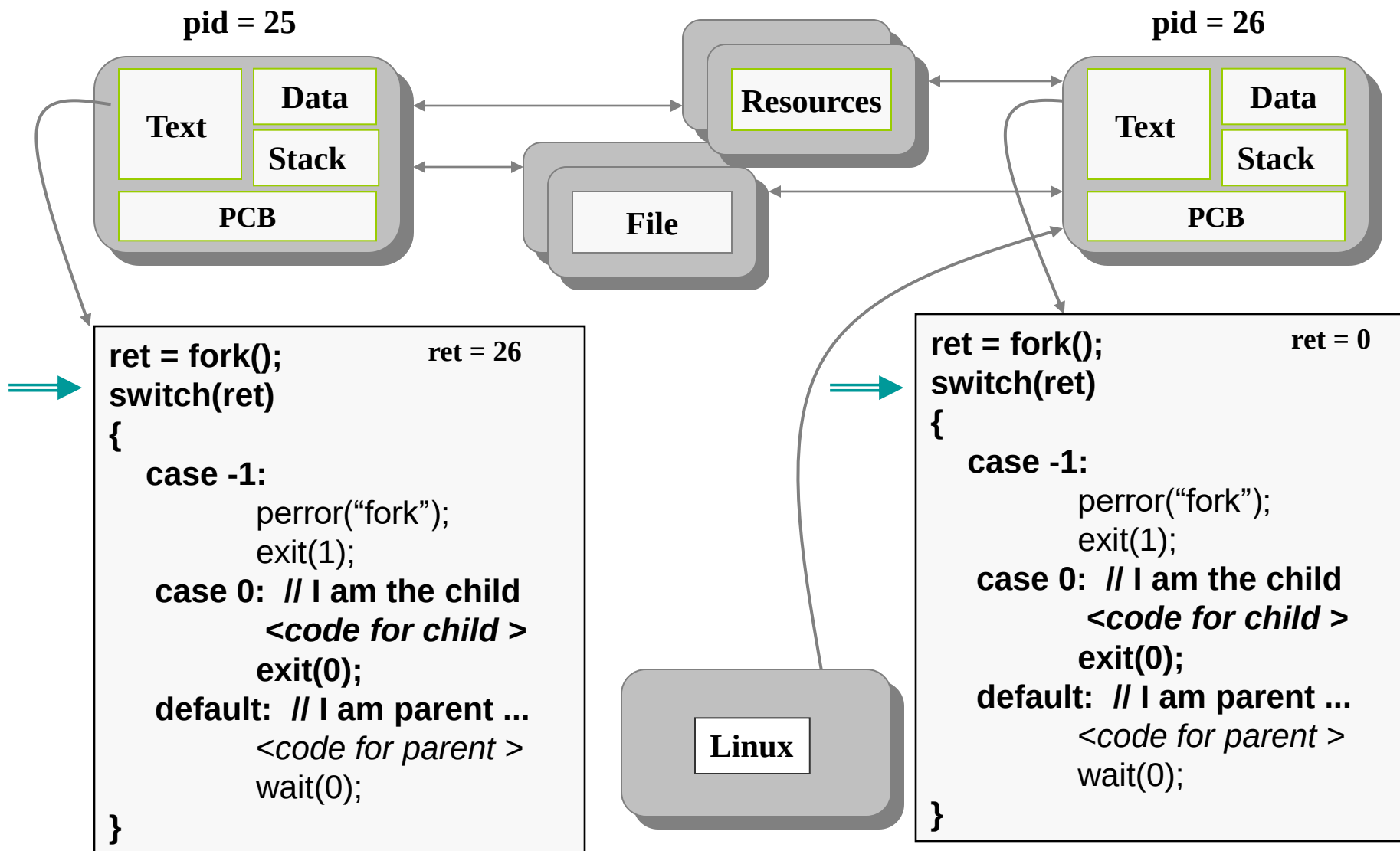
- 通过复制调用进程创建子进程
- 返回值
 - ◆ 子进程: 0
 - ◆ 父进程: 子进程ID
- 出错: -1



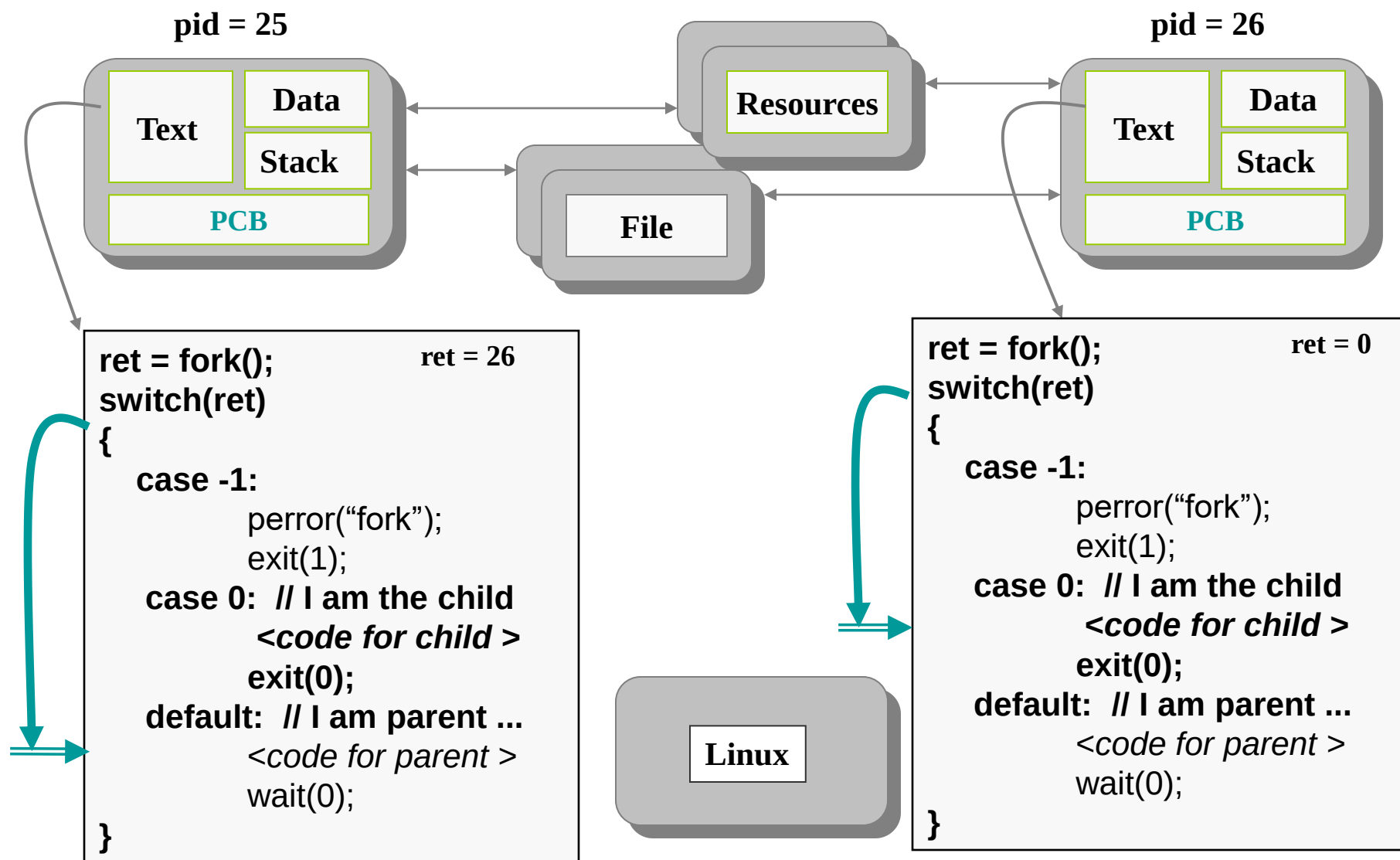
fork 是如何工作的？(1)



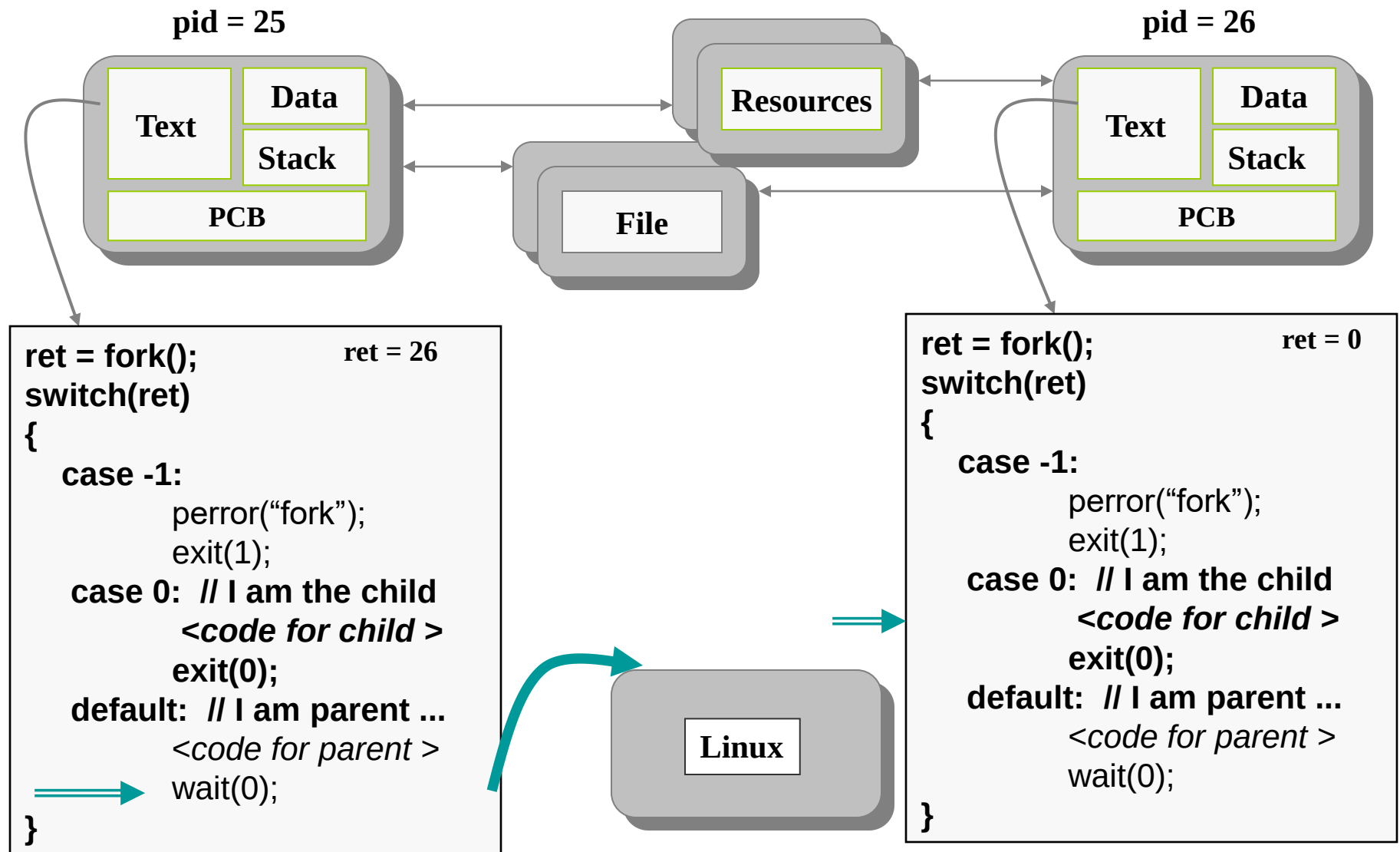
fork 是怎么工作的？ (2)



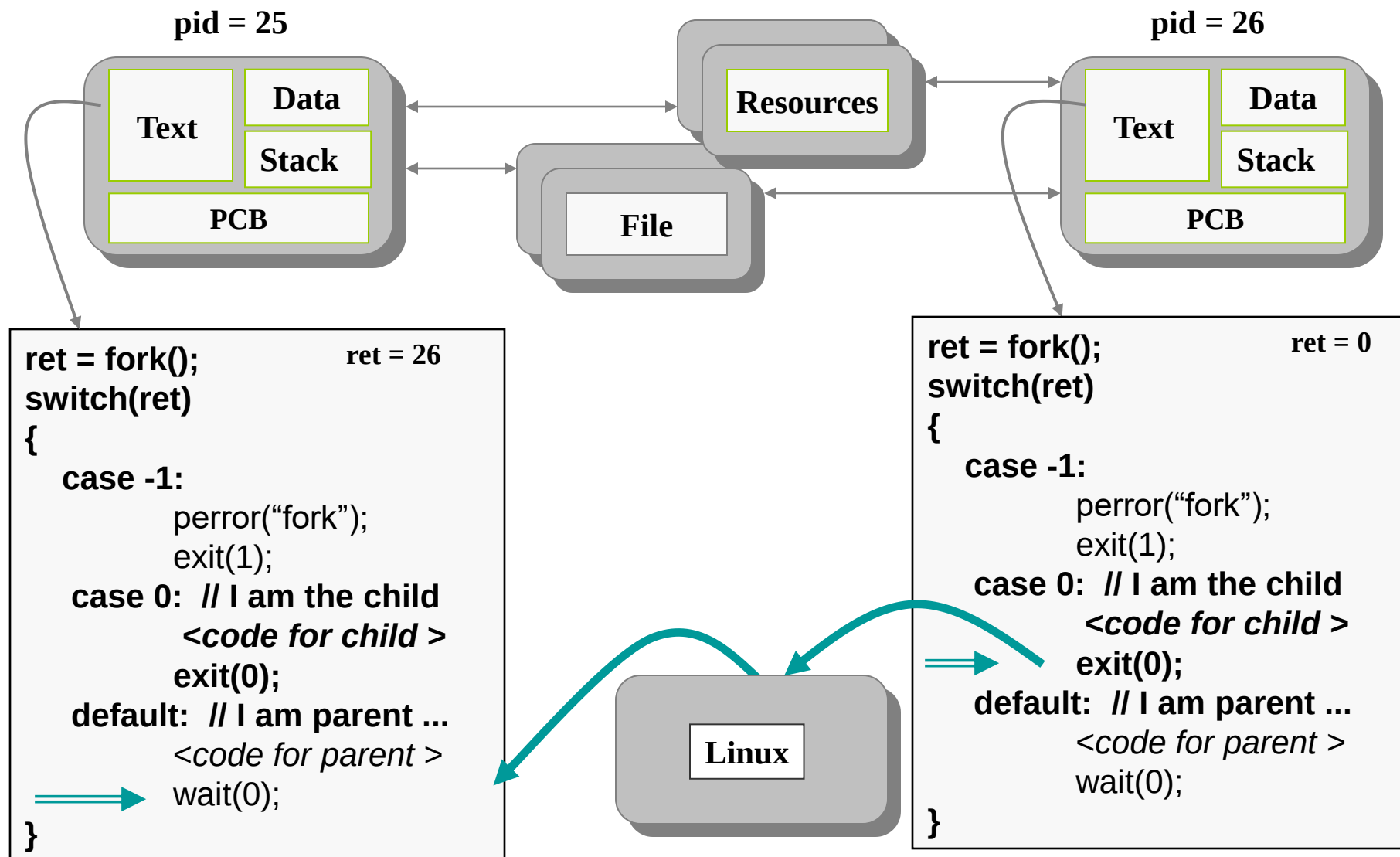
fork 是怎么工作的？ (3)



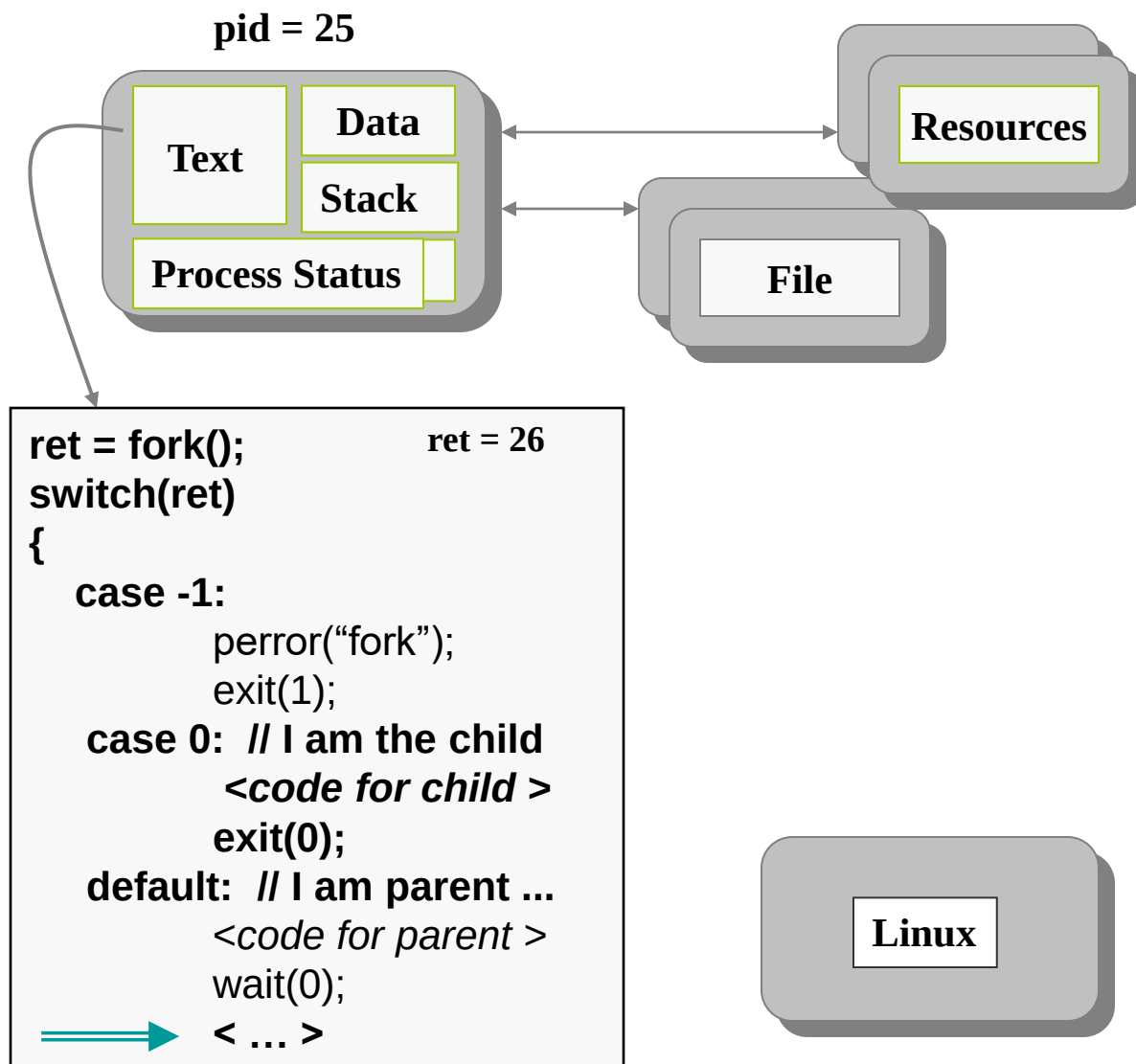
fork 是怎么工作的？ (4)



fork 是怎么工作的? (5)



fork 是怎么工作的？ (6)



创建一个新的进程 - fork()

■ 系统调用fork() 出错返回-1

- errno = EAGAIN

- ◆ 内存不足拷贝父进程的页表和创建PCB结构

- ◆ 进程数超过了用户进程数限制

- errno = ENOMEM

- ◆ 内存不足以构建内核数据结构

- errno = ENOSYS

- ◆ 系统平台不支持

■ 查看errno数值的意义

- errno.h

- man 3 errno

fork() – 例1

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>

int main(){
    pid_t  childpid;

    childpid = fork();
    if (childpid == -1){
        perror("Failed to fork");
        exit(1);
    }
    if (childpid == 0)
        printf("I am the child process with ID: %lu \n",
            (long)getpid());
    else
        printf("I am the parent process with ID: %lu \n",
            (long)getpid());
    return 0;
}
```

fork() – 例2

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>

int main(){
    pid_t  childpid;
    pid_t  mypid;

    mypid = getpid();
    childpid = fork();
    if (childpid == -1){
        perror("Failed to fork");
        exit(1);
    }
    if (childpid == 0)
        printf("I am the child process with ID: %lu -- %lu\n",
            (long)getpid(), (long)mypid);
    else { sleep(2);
        printf("I am the parent process with ID: %lu -- %lu\n",
            (long)getpid(), (long)mypid); }
    return 0;
}
```

fork() - 例3

```
#include <stdio.h>
#include <stdlib.h>

int main(void){
    pid_t pid;

    printf("Original process: PID = %d, PPID = %d\n",getpid(), getppid());
    pid = fork();
    if ( pid == -1 ){
        perror("fork"); exit(1);
    }
    if ( pid != 0 )
        printf("Parent process: PID = %d, PPID = %d, CPID = %d \n",
            getpid(), getppid(), pid);
    else {
        sleep(2);
        printf("Child process: PID = %d, PPID = %d \n",
            getpid(), getppid());
    }
    printf("Process with PID = %d terminates \n",getpid());
}
```

```
Original process: PID = 16816, PPID = 3256
Parent process: PID = 16816, PPID = 3256, CPID = 16817
Process with PID = 16816 terminates
Child process: PID = 16817, PPID = 1
Process with PID = 16817 terminates
```